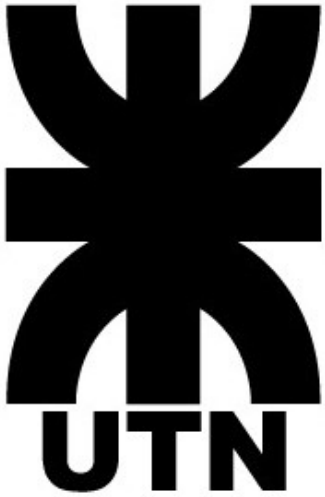
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL MENDOZA



Ingeniería y Calidad de Software
Trabajo Práctico N°7

GRUPO Nº4 - 4K10
MÓDULO 1 Y 4

Integrantes:

- Barchiesi Santino 50791
- Estudillo Máximo 50013
- Navarro Matías 51006

Repositorio y organización del equipo 3
Ramas, versiones y commits 3
Roles del equipo 5
Incidente y defecto 6
El incidente 6
Análisis del síntoma 6
El defecto: no existía, lo inyectamos 7
Por qué llegó a producción 9
Impacto 10
La corrección 10
Plan de desarrollo y mantenimiento 11
Tiempos previstos 11
1. Gestión del cambio 11
2. Estrategia de ramas 12
3. Proceso de revisión 13
4. Aseguramiento de la calidad 14
5. Ambientes 15
6. Despliegue y aprobación 16
7. Plan de reversión 17
8. Comunicación y cierre 18
Pipeline con GitHub Actions 20
Diseño 20
El workflow 21
Formato, linter y build 22
Tests que reproducen el defecto 23
Protección de ramas 26
Ejecuciones del pipeline 27
Consistencia entre el plan y el pipeline 32
Justificación de los niveles del pipeline 34
Por qué no abordamos el nivel avanzado 34
Lecciones aprendidas 36
Uso de IA 39

Repositorio y organización del equipo

Repositorio GitHub del equipo: https://github.com/santinobarchiesi2003/agendaya-tp7-grupo4

El repositorio parte de la aplicación y los tests del TP6 (https://github.com/maxidoe2/agendaya-tp6-grupo4). Sobre esa base agregamos el control de formato, el linter, el build, el pipeline de GitHub Actions y las plantillas de Pull Request y de incidente. Después simulamos el último despliegue con la versión 1.1.0, que trae el defecto del incidente, y lo resolvimos en una rama hotfix siguiendo el plan de este informe.

Qué	Dónde
Pipeline de GitHub Actions	.github/workflows/ci.yml
Reglas de formato (Prettier)	.prettierrc.json, .prettierignore y .gitattributes
Reglas del linter (ESLint)	eslint.config.js
Build	scripts/build.js
Tests unitarios (Jest)	tests/unit: 6 archivos, 49 tests
Tests E2E (Cypress)	cypress/e2e: 6 archivos, 12 tests
Plantillas de Pull Request y de incidente	.github/pull_request_template.md y .github/ISSUE_TEMPLATE/incidente.md

Ramas, versiones y commits

Trabajamos con dos ramas permanentes, main y develop, y una rama corta para el hotfix. Cada versión que se despliega lleva un tag.

Rama o tag	Qué es
main	Lo que está en producción.
develop	Integración de la próxima versión.
hotfix/INC-0452-confirmacion-zona-horaria	La corrección del incidente. Nace de main en v1.1.0.
v1.0.0	La aplicación del TP6 con el pipeline. Es la última versión sin el defecto.
v1.1.0	El último despliegue. Trae el defecto.
v1.1.1	El hotfix, sobre el commit de merge en main.

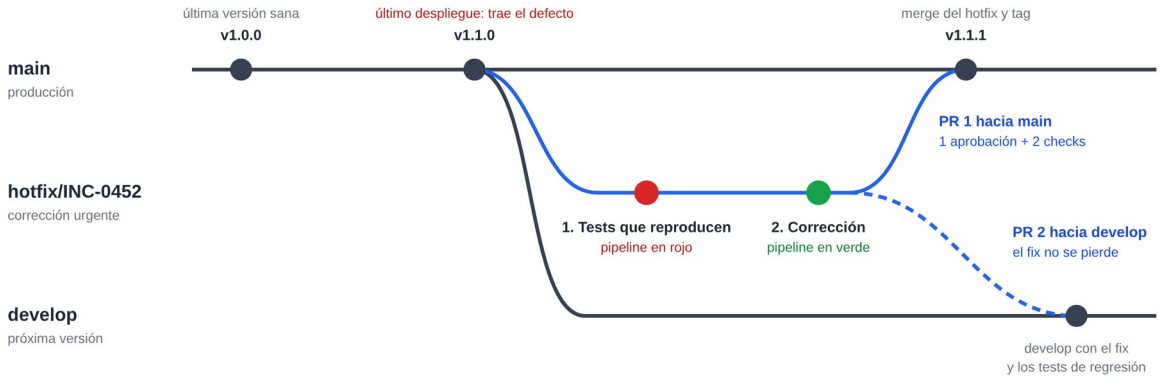


Figura 1 – Recorrido del hotfix entre las ramas.

Los commits, en orden:

N.º	Commit	Qué hace
1	Partir de la aplicacion y los tests del TP6 como base del TP7	Copia el servidor, las pantallas, la lógica y los tests del TP6.
2	Agregar Prettier para controlar el formato del codigo	Configuración, scripts y .gitattributes.
3	Formatear el codigo existente con Prettier	Cambio mecánico: 13 archivos, sin cambios de comportamiento.
4	Agregar ESLint y corregir lo que marca	Configuración y corrección de los 3 errores que marcó.
5	Agregar el build: paquete versionado de la aplicacion	Script de build y npm run verificar.
6	Pipeline de CI con GitHub Actions y plantillas de PR e incidente	El workflow y las dos plantillas.
7	README del TP7: verificaciones, pipeline y estrategia de ramas	Documentación. Tag v1.0.0.
8	Confirmacion: mostrar el horario de inicio y de fin del turno (v1.1.0)	El último despliegue. Introduce el defecto. Tag v1.1.0.
9	Tests que reproducen INC-0452: confirmacion corrida una hora	Rama hotfix. El pipeline falla.
10	Corregir INC-0452: la confirmacion usa el horario de la agenda (v1.1.1)	Rama hotfix. El pipeline pasa.

Roles del equipo

Somos tres, así que cada integrante cubre un rol y tiene un suplente. Hay una regla que no se rompe en ningún caso: quien escribe una corrección no la revisa ni aprueba su despliegue.

Rol	Integrante	Qué hace en un hotfix	Suplente
Tech Lead	Barchiesi Santino	Clasifica el incidente y autoriza el cambio. Aprueba el despliegue a producción. Comunica el avance y cierra el incidente. Administra el repositorio.	Estudillo Máximo
Desarrollador de guardia de M04	Navarro Matías	Busca la causa, escribe el test que reproduce el defecto y la corrección, abre los Pull Request y ejecuta el despliegue.	Estudillo Máximo. En ese caso revisa Barchiesi Santino.
Revisor y QA	Estudillo Máximo	Revisa y aprueba el Pull Request. Hace la validación manual en staging.	Barchiesi Santino

Soporte N1 no forma parte del equipo: registra el incidente y es el contacto con los usuarios.

En la ejecución de este TP el repositorio quedó en la cuenta de GitHub de Santino y los comandos se corrieron desde su computadora. Por eso el issue y los dos Pull Request figuran abiertos por su usuario. Los commits del hotfix son de Matías y las dos aprobaciones son de Máximo, así que la regla se cumplió: quien escribió la corrección no la aprobó.

Incidente y defecto

El incidente

El incidente lo definimos nosotros. Usamos como modelo el ejemplo del enunciado, que es de nuestro módulo (M04 Booking Público), y lo escribimos para nuestra implementación, como lo reportaría Soporte.

Campo	Valor
Identificador	INC-0452
Severidad	Crítica
Módulo	M04 Booking Público
Síntoma	Desde el despliegue del viernes, invitados que reservan con el celular desde fuera de Argentina avisan que la pantalla de confirmación les muestra el turno una hora antes del horario que eligieron. El correo de confirmación les llega con otro horario y preguntan cuál de los dos vale.
Impacto	14 consultas de invitados desde el sábado. No se sabe cuántas reservas más están afectadas.
Reportado por	Soporte N1
Detectado	Lunes 10:15
SLA de resolución	4 horas. Vence el lunes a las 14:15.

Análisis del síntoma

El incidente está descripto como lo ve un usuario. Antes de tocar el código comparamos cada parte del reporte con nuestra implementación:

Lo que dice el reporte	Lo que encontramos
"Desde el despliegue del viernes"	El despliegue del viernes es la versión 1.1.0, que cambió una sola cosa: la confirmación de la reserva pasó a mostrar el horario de inicio y de fin del turno. Ahí empezamos a buscar.

Lo que dice el reporte	Lo que encontramos
"Reservan con el celular"	No hay código distinto para el celular: es la misma pantalla (reservar.html) con una regla de estilo para pantallas angostas. El celular no es la causa. Los reportes vienen de ahí porque es el dispositivo desde el que reserva alguien que está de viaje o vive en otro país, y el celular cambia su zona horaria solo.
"Desde fuera de Argentina"	Es la variable que importa. Lo que cambia afuera es la zona horaria del dispositivo. Con el dispositivo en la zona de la agenda (Mendoza, UTC-3) la confirmación sale bien. Con otra zona sale corrida.
"Una hora antes"	El corrimiento es la diferencia entre la zona del dispositivo y la de la agenda. Una hora antes corresponde a un dispositivo en UTC-4, por ejemplo en Bolivia. Desde Perú o Colombia (UTC-5) son dos horas.
"El correo les llega con otro horario"	Es la pista más útil. El correo lo arma el servidor y la confirmación de la pantalla la arma el navegador del invitado. Comprobamos que el correo y la reserva guardada tienen el horario de la agenda, así que el problema está en el código que corre en el navegador.

El defecto: no existía, lo inyectamos

El defecto no existía en nuestra implementación. Hasta el TP6 la confirmación se armaba con el día y la hora tal como están guardados, como texto ("2026-10-13" y "10:00"), sin convertir nada. La zona horaria del dispositivo no influía.

Lo introdujimos en main con el commit "Confirmacion: mostrar el horario de inicio y de fin del turno (v1.1.0)", que simula el último despliegue. Es una mejora chica y creíble: que la confirmación diga también hasta qué hora es el turno.

Versión	Texto de la confirmación
1.0.0	Juan Perez, te esperamos el 2026-10-13 a las 10:00 h.
1.1.0	Juan Perez, te esperamos el 2026-10-13 de 10:00 a 11:00 h.

Para sumarle la duración al horario de inicio, la función nueva convierte el turno en un objeto Date. Este es el código de la versión 1.1.0 (src/logica/reservas.js):

```
const MS_POR_MINUTO = 60 * 1000;

/**
 * Desfase de la agenda respecto de UTC. La agenda del administrador esta en
 * hora de Mendoza: UTC-3 todo el anio (Argentina no usa horario de verano).
```

```
 */
const DESFASE_AGENDA = "-03:00";

/** Hora "HH:MM" de un instante. */
function horaDe(instante) {
  return (
    String(instante.getHours()).padStart(2, "0") + ":" +
String(instante.getMinutes()).padStart(2, "0")
  );
}

function textoConfirmacion(reserva, duracionMin) {
  const duracion = duracionMin || DISPONIBILIDAD.duracionMin;
  const inicio = new Date(reserva.fecha + "T" + reserva.hora + ":00" +
DESFASE_AGENDA);
  const fin = new Date(inicio.getTime() + duracion * MS_POR_MINUTO);
  return (
    reserva.nombre +
    ", te esperamos el " +
    reserva.fecha +
    " de " +
    horaDe(inicio) +
    " a " +
    horaDe(fin) +
    " h."
  );
}
```

Qué produce el defecto. La primera mitad está bien: new Date("2026-10-13T10:00:00-03:00") es el mismo instante en cualquier dispositivo. El error está en cómo se muestra: getHours() y getMinutes() devuelven la hora de ese instante según la zona horaria configurada en el dispositivo donde corre el código. Ese código corre en el navegador del invitado, así que el horario que se muestra depende de dónde esté el invitado.

Zona horaria del dispositivo	Confirmación del turno de las 10:00
Mendoza (UTC-3), la de la agenda	de 10:00 a 11:00 h. Correcto.
La Paz (UTC-4)	de 09:00 a 10:00 h. Es el corrimiento de una hora del incidente.
Lima o Bogotá (UTC-5)	de 08:00 a 09:00 h.
UTC, la del runner de GitHub Actions	de 13:00 a 14:00 h.

En la pantalla de reserva se ve la contradicción (Figura 2, centro): la confirmación dice de 09:00 a 10:00 y, justo debajo, la agenda marca como reservado el turno de las 10:00.

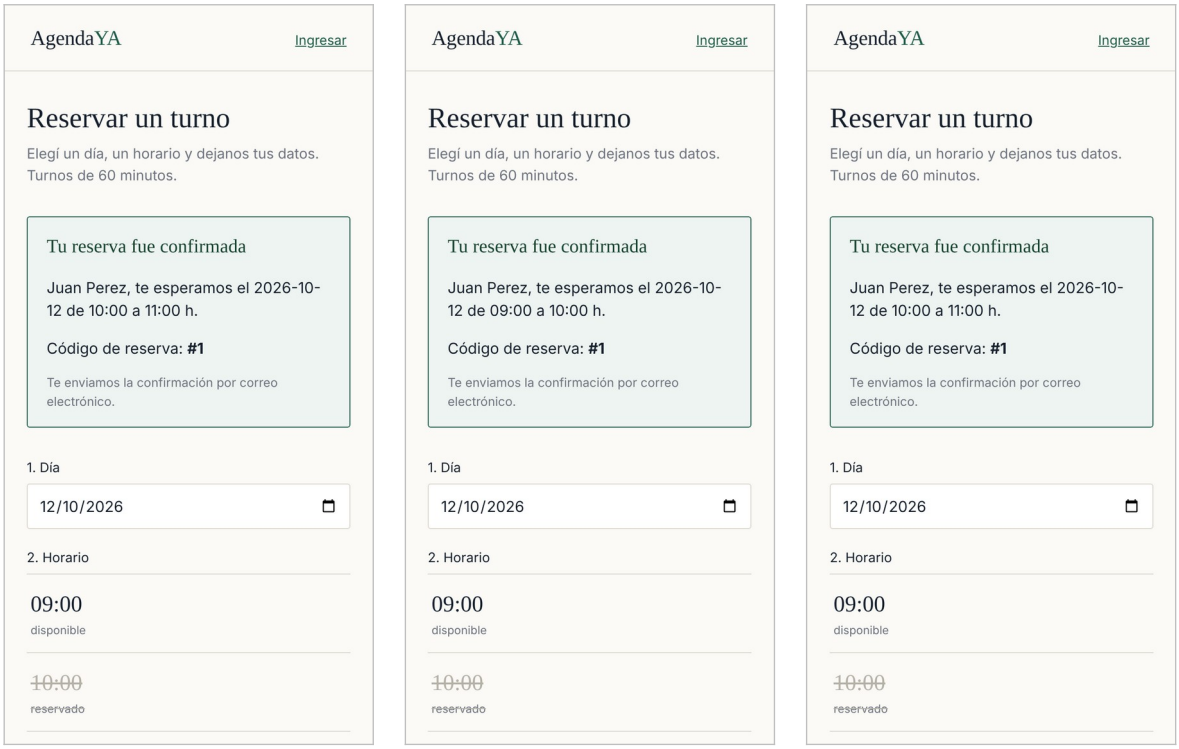


Figura 2 – La misma reserva del turno de las 10:00 en un navegador con tamaño de teléfono. Izquierda: versión 1.1.0 con la zona horaria de la agenda. Centro: versión 1.1.0 con la zona de La Paz, emulada en el navegador. Derecha: versión 1.1.1, ya corregida, con la zona de La Paz.

Por qué llegó a producción

El pipeline pasó en verde con el defecto adentro (Figura 3). Encontramos tres motivos:

- **La función nueva entró sin tests.** Ningún test unitario llamaba a textoConfirmacion.
- **El test E2E miraba el día pero no la hora.** m04-reserva-exitosa verifica que la confirmación contenga la fecha del turno. Nunca verificó el horario.
- **En nuestras computadoras el defecto no se ve.** Están configuradas con la misma zona horaria que la agenda, y ahí la confirmación sale bien. En el runner de GitHub, que está en UTC, la confirmación sale de 13:00 a 14:00, pero ningún test miraba la hora.

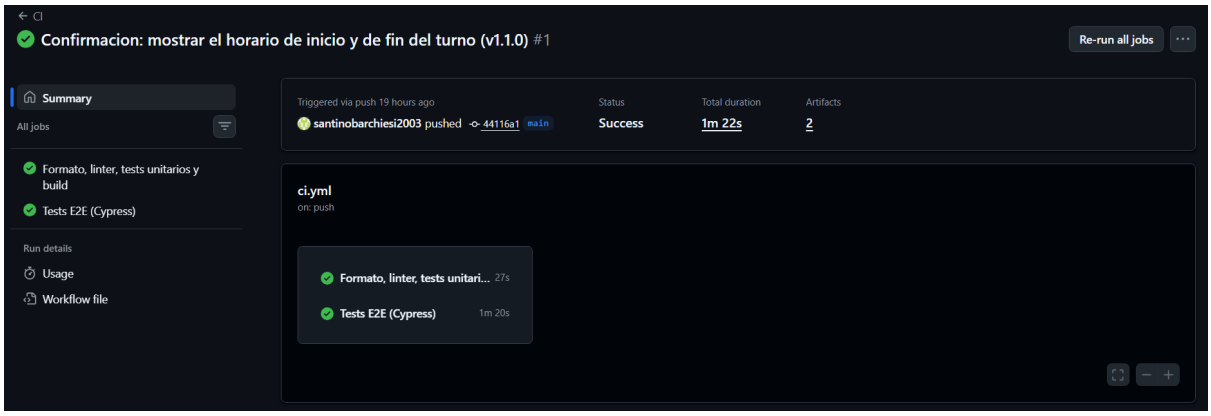


Figura 3 – Ejecución del pipeline sobre main con la versión 1.1.0, disparada por el push: los dos jobs en verde con el defecto adentro.

Impacto

Qué	Estado con la versión 1.1.0
Reserva guardada en el sistema	Correcta: se guardan el día y la hora de la agenda.
Correo de confirmación al invitado	Correcto: lo arma el servidor con el horario guardado.
Aviso al administrador y panel de reservas	Correctos.
Confirmación en la pantalla del invitado	Incorrecta si el dispositivo está en otra zona horaria.

Soporte recibió 14 consultas, pero las reservas afectadas pueden ser más: cualquier reserva hecha desde un dispositivo en otra zona horaria a partir del viernes. El riesgo para el invitado es presentarse a una hora que no es la de su turno. Como el correo siempre tuvo el horario correcto, eso es lo que Soporte les indica a los invitados mientras se resuelve el incidente (ver Comunicación y cierre).

La corrección

La corrección es la versión 1.1.1. El texto usa la hora de la agenda tal como está guardada, y el horario de fin se calcula sumando minutos con dos funciones que el módulo ya tenía (aMinutos y aHora). No pasa por Date, así que la zona horaria del dispositivo deja de influir.

```
/**
 * Texto con el que se le confirma la reserva al invitado, con el horario de
 * inicio y de fin del turno:
 *
 *   "Juan Perez, te esperamos el 2026-10-13 de 10:00 a 11:00 h."
 *
 * El horario es el de la agenda, tal como lo eligio el invitado, y el fin se
 * calcula sumando minutos. No se usa Date: getHours() y getMinutes() responden
 * segun la zona horaria del dispositivo, y un invitado en otra zona veia el
 * turno corrido (INC-0452).
 */
function textoConfirmacion(reserva, duracionMin) {
  const duracion = duracionMin || DISPONIBILIDAD.duracionMin;
  const fin = aHora(aMinutos(reserva.hora) + duracion);
  return (
    reserva.nombre + ", te esperamos el " + reserva.fecha + " de " + reserva.hora + "
a " + fin + " h."
  );
}
```

Al reemplazar la función, el linter marcó como sin uso las constantes DESFASE_AGENDA y MS_POR_MINUTO y la función horaDe. Las borramos en el mismo commit. En total el hotfix cambia un archivo de código (src/logica/reservas.js: 6 líneas agregadas y 25 quitadas), agrega tres archivos de tests y sube la versión en package.json. Con la corrección, el mismo teléfono en la zona horaria de La Paz ve el horario que eligió (Figura 2, derecha).

Plan de desarrollo y mantenimiento

El plan describe cómo se gestiona un cambio urgente sobre producción y cómo llega al ambiente productivo con los mismos controles que un cambio normal, aunque esté corriendo el SLA. Vale para cualquier hotfix. En cada sección indicamos cómo se aplica a INC-0452 y qué pasa en el caso excepcional.

Tiempos previstos

El SLA de INC-0452 es de 4 horas desde la detección: del lunes a las 10:15 al lunes a las 14:15. Repartimos ese tiempo en etapas, cada una con un límite. Quedan 30 minutos de margen.

Etapa	Responsable	Límite	Hora
Registro y clasificación	Soporte N1 y Tech Lead	25 min	10:40
Causa identificada y test que la reproduce, en rojo	Desarrollador	1 h	11:15
Corrección con el pipeline en verde	Desarrollador	1 h 45 min	12:00
Revisión, aprobación y merge a main	Revisor	2 h 15 min	12:30
Validación manual en staging	QA	2 h 45 min	13:00
Aprobación y despliegue a producción	Tech Lead y desarrollador	3 h	13:15
Verificación en producción y aviso de resolución	Desarrollador y Tech Lead	3 h 30 min	13:45
Vencimiento del SLA		4 h	14:15

Una ejecución completa del pipeline tardó entre 56 segundos y 1 minuto 25 segundos en este TP, así que entra varias veces en el tiempo previsto para la corrección.

1. Gestión del cambio

¿Quién registra, clasifica y autoriza el cambio? ¿Con qué criterio se decide que es un hotfix y no un cambio ordinario?

Registro. Soporte N1 registra el incidente como issue del repositorio, con la plantilla "Incidente en producción", dentro de los 10 minutos de detectado. La plantilla pide identificador, severidad, módulo, síntoma, impacto, hora de detección y vencimiento del SLA. El SLA se cuenta desde la detección, no desde el registro. Si Soporte no tiene acceso al repositorio, lo carga el Tech Lead con los datos que le pasa Soporte.

Clasificación. La hace el Tech Lead en el mismo issue, dentro de los 15 minutos del registro. Un cambio es un hotfix solo si cumple las cuatro condiciones:

1. El defecto está en producción (existe en main), no solo en develop.
2. La severidad es crítica o alta: afecta un flujo principal (iniciar sesión, reservar) o le da al usuario información incorrecta, y el usuario no tiene forma de evitarlo.
3. El SLA vence antes de la próxima versión planificada.
4. La corrección se puede acotar al defecto, sin rediseño ni funcionalidad nueva.

Si falta alguna de las tres primeras es un cambio ordinario: rama feature/* desde develop, y sale en la próxima versión. Si se cumplen las tres primeras pero la corrección no se puede acotar, tampoco se fuerza un hotfix: se vuelve a la última versión sana (sección 7) y la corrección sigue el ciclo normal.

Autorización. El Tech Lead autoriza el hotfix con un comentario en el issue, donde nombra al desarrollador y al revisor. Sin ese comentario no se crea la rama.

Punto de control a los 60 minutos. Si una hora después de la detección no hay una causa identificada con un test que la reproduzca, el Tech Lead ordena volver a la última versión sana para cortar el daño, y la investigación sigue sin el SLA encima.

Caso excepcional. Si el Tech Lead no responde en 15 minutos, clasifica y autoriza su suplente. Quien autoriza no puede ser quien va a escribir la corrección.

En INC-0452:

Condición	Se cumple	Por qué
1. Está en producción	Sí	Está en la versión 1.1.0, que es la desplegada.
2. Severidad crítica o alta	Sí	El invitado recibe un horario equivocado y no tiene cómo darse cuenta.
3. El SLA vence antes de la próxima versión	Sí	Vence en 4 horas.
4. La corrección se puede acotar	Sí	Es una sola función, textoConfirmacion.

Decisión: hotfix. A los 60 minutos había causa y test, así que no hizo falta volver a la versión anterior.

2. Estrategia de ramas

¿Desde qué rama nace el hotfix, hacia dónde se integra y cómo se evita que el fix se pierda en la próxima versión?

Rama	Para qué	Nace de	Vuelve a
main	Lo que está en producción. Cada despliegue lleva un tag.	—	—
develop	Integración de la próxima versión.	main	main
feature/*	Un cambio que sale en la próxima versión.	develop	develop
hotfix/*	Corrección urgente de un defecto que está en producción.	main	main y develop

De dónde nace. De main, en el tag de la versión que está en producción (v1.1.0). Nunca de develop: develop tiene trabajo de la próxima versión que todavía no se liberó y saldría a producción junto con el fix. El nombre es hotfix/ seguido del identificador del incidente y una descripción corta: hotfix/INC-0452-confirmacion-zona-horaria.

Qué commits lleva. Dos. Primero el test que reproduce el defecto, con el que el pipeline tiene que fallar. Después la corrección, con la versión nueva. Un hotfix sube el tercer número de la versión: de 1.1.0 a 1.1.1.

Hacia dónde se integra. Primero a main, por Pull Request. Es el único camino: main está protegida y no acepta push directo, tampoco de los administradores. Se integra con un commit de merge, sin squash, para que los dos commits queden en el historial. Sobre el commit de merge se crea el tag v1.1.1.

Cómo se evita que el fix se pierda.

1. Apenas se mergea a main se abre un segundo Pull Request, de la misma rama hacia develop, con el mismo pipeline y la misma revisión.
2. El incidente no se cierra hasta que ese Pull Request está mergeado. Es un ítem de la lista de seguimiento del issue.
3. El test de regresión viaja con el fix. Si en una versión futura el defecto vuelve, o el fix se pierde en un merge, el pipeline falla.
4. La rama hotfix se borra después de los dos merges, no antes.

Caso excepcional. Si develop modificó la misma parte del código y el Pull Request hacia develop tiene conflictos, los resuelve el autor del hotfix en ese mismo Pull Request, y el revisor verifica que el test de regresión siga estando y pase. Si no se puede resolver en el día, el Pull Request queda abierto y la próxima versión no sale mientras siga abierto.

3. Proceso de revisión

¿Quién revisa, cuántas aprobaciones se requieren y qué se revisa? ¿Qué pasa si el revisor no está disponible dentro del SLA?

Quién. Un integrante que no haya escrito la corrección. El revisor designado es Estudillo Máximo y su suplente es Barchiesi Santino.

Cuántas aprobaciones. Una. Está configurado en la protección de main y de develop: una aprobación requerida, que se descarta si entra un commit nuevo. GitHub no permite que el autor apruebe su propio Pull Request.

Qué se revisa. La lista está en la plantilla de Pull Request:

1. El cambio se limita al incidente: no hay refactors, mejoras ni cambios de formato ajenos.
2. Hay un test nuevo que reproduce el defecto. En el historial del Pull Request están la ejecución en rojo anterior al fix y la ejecución en verde posterior.
3. No se modificó ni se borró ningún test existente para que el pipeline pase.
4. Los dos checks del pipeline están en verde en el último commit.
5. La versión subió el tercer número y el Pull Request dice cómo revertir.

Tiempos. La revisión empieza dentro de los 15 minutos de pedida y dura 30 como máximo. Si el revisor pide cambios, el autor los sube, el pipeline corre de nuevo completo y hace falta aprobar otra vez.

Caso excepcional. Si el revisor no contesta en 15 minutos, el autor le avisa al Tech Lead, que revisa él, siempre que no haya participado de la corrección. Si a los 30 minutos no hay un segundo integrante disponible, el hotfix no se mergea sin revisión y tampoco se usa un permiso de administrador, que está deshabilitado. En ese caso el desarrollador vuelve a la última versión sana (sección 7): esa versión ya fue revisada y estuvo en producción, así que no necesita una revisión nueva. El hotfix espera al revisor.

4. Aseguramiento de la calidad

¿Qué pruebas se ejecutan, cuáles son automatizadas y cuáles manuales? ¿Cómo se verifica que el fix no rompió otra funcionalidad (regresión)?

Prueba	**Tipo**	**Cuándo**	**Qué cubre**
Formato (Prettier)	Automatizada	En cada push al Pull Request	Que el código siga el formato del proyecto.
Linter (ESLint)	Automatizada	En cada push al Pull Request	Errores comunes, variables sin definir y código sin uso, también en los scripts de las pantallas.
Tests unitarios (Jest)	Automatizada	En cada push al Pull Request	Las reglas de M01 y M04: 49 tests, incluidos los del incidente.
Build	Automatizada	En cada push al Pull Request	Que el paquete de la aplicación se arme completo.

Prueba	Tipo	Cuándo	Qué cubre
Tests E2E (Cypress)	Automatizada	En cada push al Pull Request	Los flujos de registro, inicio de sesión y reserva de punta a punta: 12 tests, incluido el escenario del incidente.
Escenario del incidente en un teléfono real	Manual	En staging, antes de aprobar el despliegue	Lo que la emulación no cubre: un navegador móvil real con otra zona horaria.
Recorrido de los flujos principales	Manual	En staging y después del despliegue	Registro, inicio de sesión, reserva y panel.

Regresión. En cada push corre la batería completa, no solo el test nuevo: los 45 tests unitarios y los 10 E2E que ya existían tienen que seguir pasando. El revisor verifica además que ninguno se haya modificado. La protección de rama exige los dos jobs en verde para habilitar el merge.

Test que reproduce el defecto. Todo hotfix incluye al menos un test que falla sin la corrección y pasa con ella. Tiene que quedar una ejecución del pipeline en rojo con ese test antes del commit de la corrección.

Validación manual en staging. La hace QA en 15 minutos y deja el resultado escrito en el issue:

1. Con un teléfono, cambiar la zona horaria a una distinta de la agenda (por ejemplo La Paz) y reservar un turno. La confirmación tiene que mostrar el horario elegido.
2. Repetir con la zona horaria de la agenda.
3. En los dos casos, el correo y el panel del administrador muestran el mismo horario.
4. Registrar un administrador e iniciar sesión.

Caso excepcional. Si falla un test que no tiene relación con el cambio, se vuelve a ejecutar una sola vez. Si falla de nuevo se trata como una falla real: no se deshabilita el test ni se saltea el check. Si QA encuentra un problema en staging, el hotfix vuelve al desarrollador y repite el camino completo.

5. Ambientes

¿Por qué ambientes pasa el cambio antes de llegar a producción y qué se valida en cada uno?

Ambiente	Qué corre	Qué se valida	Quién
Desarrollo	La rama hotfix en la computadora del desarrollador, con los datos de prueba.	Que el test nuevo falla y, con la corrección, pasa. npm run verificar y npm run test:e2e.	Desarrollador

Ambiente	Qué corre	Qué se valida	Quién
Integración continua	El commit del Pull Request en un runner de GitHub Actions: máquina nueva, Ubuntu, Node 22, zona horaria UTC.	Formato, linter, tests unitarios, build y tests E2E, sin depender de la computadora del autor.	Automático. El resultado lo mira el revisor.
Staging	El paquete que generó el pipeline, con la misma configuración que producción y datos de prueba.	La validación manual de la sección 4.	QA
Producción	El mismo paquete que se validó en staging.	Una reserva de prueba desde un dispositivo en otra zona horaria, y 30 minutos de observación.	Desarrollador y Tech Lead

El mismo paquete en todos lados. Lo que se instala en staging y en producción es el paquete que generó el pipeline sobre el commit de merge en main. No se vuelve a armar a mano.

Staging igual a producción. Para este incidente importa en particular la zona horaria del servidor, que tiene que ser la de la agenda (America/Argentina/Mendoza): el servidor usa su reloj para decidir qué turnos ya pasaron.

develop viene después. Un hotfix no pasa por develop antes de llegar a producción. Llega a develop después, con el segundo Pull Request.

Estado actual. Los dos primeros ambientes existen y están automatizados. AgendaYA todavía no tiene servidores de staging ni de producción, porque la aplicación corre en nuestras computadoras. Para esos dos ambientes el plan define qué se hace y quién lo hace, y el paso es manual.

Caso excepcional. Si staging no está disponible, la validación manual no se saltea: QA instala el paquete en su computadora con la configuración de producción, hace la misma lista y lo deja anotado en el issue.

6. Despliegue y aprobación

¿El despliegue a producción es automatizado o manual? ¿Requiere aprobación? ¿De quién? ¿Cómo queda registrada?

Es manual y requiere aprobación. No hay despliegue automático a producción. Lo aprueba el Tech Lead y lo ejecuta el desarrollador de guardia.

Qué verifica el Tech Lead antes de aprobar:

1. El Pull Request está mergeado a main con la aprobación del revisor.
2. El pipeline está en verde sobre el commit de merge en main.

3. QA dejó anotado en el issue que la validación en staging pasó.
4. Está identificada la versión a la que se vuelve si algo sale mal.

Cómo queda registrada. La aprobación es un comentario del Tech Lead en el issue del incidente ("Aprobado el despliegue de v1.1.1 a producción"): GitHub guarda quién lo escribió y cuándo. El despliegue queda registrado con el tag v1.1.1 sobre el commit de merge, y con la hora y el resultado de la verificación anotados en el mismo issue.

Pasos del despliegue:

1. Descargar el paquete (paquete-agendaya) de la ejecución del pipeline sobre el commit de merge en main.
2. Instalarlo en producción y reiniciar el servicio.
3. Hacer una reserva de prueba desde un dispositivo en otra zona horaria y verificar la confirmación.
4. Observar 30 minutos: que se sigan creando reservas y que Soporte no reciba reportes nuevos.
5. Crear el tag y anotar el resultado en el issue.

Cuándo. Apenas está aprobado. Un hotfix no espera una ventana de despliegue.

Caso excepcional. Si el Tech Lead no está disponible, aprueba su suplente. No cambia la regla: quien aprueba el despliegue no es quien escribió la corrección.

7. Plan de reversión

Si el hotfix falla en producción, ¿cómo se vuelve atrás y en cuánto tiempo?

Cuándo se revierte. Con cualquiera de estas señales durante los 30 minutos de observación:

- La reserva de prueba posterior al despliegue falla o la confirmación sigue saliendo mal.
- Aparece un error en un flujo principal que antes funcionaba.
- Soporte recibe un reporte nuevo que se puede atribuir al cambio.

Quién decide. El Tech Lead, en el momento, sin reunión y sin una aprobación de código nueva.

A qué versión se vuelve. A la última versión sana. En INC-0452 es la 1.0.0 y no la 1.1.0, que es la que tiene el defecto. Se pierde el horario de fin en la confirmación, que vuelve a decir "a las 10:00 h.", un texto correcto.

Cómo. Se instala el paquete de la versión 1.0.0 y se reinicia el servicio. El paquete se genera desde el tag: git checkout v1.0.0, npm ci y npm run build. No hay que revertir datos, porque ninguna versión 1.1.x cambió el formato de lo que se guarda.

En cuánto tiempo. 15 minutos desde la decisión: 5 para generar el paquete, 5 para instalarlo y reiniciar, y 5 para verificar con una reserva de prueba.

Después. Se abre un Pull Request que revierte en main el merge del hotfix, para que main vuelva a coincidir con lo que está en producción. Pasa por el pipeline y por la revisión, pero se hace con el servicio ya restablecido. El incidente se reabre y se avisa a Soporte.

Caso excepcional. Si la versión anterior tampoco funciona, el Tech Lead le avisa al responsable del producto y a Soporte que las reservas están suspendidas, y el equipo completo pasa a trabajar en el incidente.

8. Comunicación y cierre

¿A quién se notifica y en qué momento? ¿Cómo se documenta el incidente una vez resuelto?

Momento	**A quién**	**Quién avisa**	**Qué se informa**
Al clasificar, dentro de los 25 minutos	Soporte N1 y responsable del producto	Tech Lead	Que el incidente está confirmado y se resuelve con un hotfix, y la hora estimada. Qué decirle a un invitado que consulta: el horario válido es el del correo de confirmación.
Cada 60 minutos, o al cambiar de etapa	Soporte N1	Tech Lead	En qué etapa está y si la hora estimada cambió.
Antes de desplegar	Soporte N1	Tech Lead	La versión y la hora del despliegue. No hay corte del servicio.
Con la corrección verificada en producción	Soporte N1, responsable del producto y administradores con reservas afectadas	Tech Lead	Que está resuelto, en qué versión, y qué reservas conviene revisar.
El mismo día	Invitados afectados	Soporte N1	Un correo con el horario correcto de su turno.

Cómo se identifica a los afectados. No guardamos la zona horaria del dispositivo, así que no se puede saber cuáles reservas se hicieron desde otro huso. Se toman todas las reservas creadas entre el despliegue de la versión 1.1.0 y el de la 1.1.1, y a esos invitados se les reenvía la confirmación aclarando que ese es el horario válido.

Cierre. El Tech Lead cierra el issue cuando están todos los ítems de la lista de seguimiento:

- La corrección está verificada en producción.
- El Pull Request hacia develop está mergeado.
- Los invitados afectados fueron notificados.
- El informe del incidente está publicado en el issue, dentro de las 48 horas. Tiene la línea de tiempo, la causa, por qué no se detectó antes, el impacto y las acciones para que no se repita, cada una con responsable y fecha.

Acciones para que no se repita, en INC-0452:

Acción	Estado
Los tests unitarios corren con una zona horaria distinta a la de la agenda.	Hecho en el hotfix.
Hay un test E2E que reserva con el navegador en otra zona horaria y verifica el horario de la confirmación.	Hecho en el hotfix.
Ningún cambio en src/logica entra sin tests de lo que agrega. Se suma como ítem a la plantilla de Pull Request.	Próxima versión.
Mostrar la zona horaria junto al horario en la agenda, la confirmación y el correo.	Próxima versión.

Caso excepcional. Si Soporte no logra contactar a un invitado afectado antes de su turno, le avisa al administrador de esa agenda para que lo llame.

Pipeline con GitHub Actions

Diseño

El pipeline es un solo workflow, CI (.github/workflows/ci.yml), con dos jobs que corren en paralelo. Se ejecuta automáticamente ante cada Pull Request hacia main o hacia develop, y también sobre el resultado de cada merge a esas ramas.

Job	Paso	Comando	Qué verifica
Formato, linter, tests unitarios y build	Formato (Prettier)	npm run format:check	Que todo el código y la configuración estén formateados.
	Linter (ESLint)	npm run lint	Buenas prácticas de JavaScript, también en los scripts de las pantallas.
	Tests unitarios (Jest)	npm run test:unit	Las reglas de M01 y M04: 49 tests.
	Build (paquete de la aplicación)	npm run build	Que se pueda armar completo el paquete que se despliega.
Tests E2E (Cypress)	Tests E2E (levanta el servidor y corre Cypress)	npm run test:e2e	Los flujos completos en un navegador contra el servidor: 12 tests.

Decisiones de diseño:

- **Cada verificación es un paso con nombre propio.** Las cuatro del primer job corren aunque falle una anterior. Así una sola ejecución muestra todo lo que hay que corregir y se ve cuál falló.
- **Dos jobs en paralelo.** El resultado llega en el tiempo del job más lento. Con un SLA corriendo, importa.
- **Instalación limpia.** Cada job instala las dependencias con npm ci desde package-lock.json, en una máquina nueva. Lo que pasa en el pipeline no depende de la computadora de quien hizo el cambio.
- **El mismo Node que en desarrollo**, la versión 22.
- **Lo mismo se puede correr antes de subir.** npm run verificar ejecuta en orden las cuatro verificaciones del primer job.
- **Quedan guardados el paquete y la evidencia.** El paquete del build (paquete-agendaya) y las capturas y videos de Cypress (evidencia-cypress) se guardan 30 días como artefactos de la ejecución.

El workflow

Archivo .github/workflows/ci.yml:

```
# Pipeline de integracion continua de AgendaYA (TP N7, Grupo 4).
#
# Corre ante cada Pull Request hacia main o hacia develop. Los hotfix tambien
# entran por ahi: la rama hotfix/* nace de main y vuelve por PR a main y despues
# a develop (ver el plan de desarrollo y mantenimiento), asi que ningun cambio
# llega a produccion sin pasar por estas verificaciones.
#
# Los dos jobs son checks obligatorios en la proteccion de main y de develop:
# el boton de merge queda bloqueado hasta que ambos terminan bien.
name: CI

on:
  pull_request:
    branches: [main, develop]
  # Tambien sobre el resultado del merge: de aca sale el paquete que se despliega
  push:
    branches: [main, develop]
  workflow_dispatch:

permissions:
  contents: read

jobs:
  verificaciones:
    name: Formato, linter, tests unitarios y build
    runs-on: ubuntu-latest
    timeout-minutes: 10
    env:
      # Este job no abre un navegador: se saltea la descarga del binario de Cypress
      CYPRESS_INSTALL_BINARY: 0
    steps:
      - name: Descargar el codigo
        uses: actions/checkout@v6

      - name: Preparar Node.js 22
        uses: actions/setup-node@v6
        with:
          node-version: 22
          cache: npm

      - name: Instalar dependencias
        id: dependencias
        run: npm ci

      # Las cuatro verificaciones corren aunque falle una anterior, para que una
      # sola ejecucion muestre todo lo que hay que corregir.
      - name: Formato (Prettier)
        if: ${{ !cancelled() && steps.dependencias.outcome == 'success' }}
        run: npm run format:check

      - name: Linter (ESLint)
        if: ${{ !cancelled() && steps.dependencias.outcome == 'success' }}
        run: npm run lint

      - name: Tests unitarios (Jest)
        if: ${{ !cancelled() && steps.dependencias.outcome == 'success' }}
        run: npm run test:unit

      - name: Build (paquete de la aplicacion)
        id: build
        if: ${{ !cancelled() && steps.dependencias.outcome == 'success' }}
        run: npm run build

      - name: Guardar el paquete generado
```

```
      if: ${{ !cancelled() && steps.build.outcome == 'success' }}
      uses: actions/upload-artifact@v6
      with:
        name: paquete-agendaya
        path: dist/*.tgz
        if-no-files-found: error
        retention-days: 30

  e2e:
    name: Tests E2E (Cypress)
    runs-on: ubuntu-latest
    timeout-minutes: 15
    steps:
      - name: Descargar el codigo
        uses: actions/checkout@v6

      - name: Preparar Node.js 22
        uses: actions/setup-node@v6
        with:
          node-version: 22
          cache: npm

      - name: Instalar dependencias y el binario de Cypress
        run: npm ci

      - name: Tests E2E (levanta el servidor y corre Cypress)
        run: npm run test:e2e

      - name: Guardar capturas y videos de Cypress
        if: ${{ !cancelled() }}
        uses: actions/upload-artifact@v6
        with:
          name: evidencia-cypress
          path: |
            cypress/screenshots
            cypress/videos
          if-no-files-found: ignore
          retention-days: 30
```

Formato, linter y build

Formato. Usamos Prettier 3. Verifica JavaScript, HTML (también los scripts que están dentro de las pantallas), CSS, JSON, YAML y Markdown. Probamos anchos de línea de 100, 110, 115 y 120 caracteres y elegimos 110, que era el que menos cambios producía sobre el código ya escrito. El primer formateo modificó 13 archivos y fue en un commit aparte, sin otros cambios. También agregamos .gitattributes para que los finales de línea sean LF en todas las máquinas: sin eso, en Windows Git los convierte a CRLF al clonar y el control de formato falla en la computadora del desarrollador aunque pase en el pipeline.

Linter. Usamos ESLint 10 con las reglas recomendadas más eqeqeq, no-var, prefer-const y curly. Con eslint-plugin-html también analiza los scripts que están dentro de los archivos HTML, que es donde vive la lógica del frontend. Sobre el código del TP6 marcó 3 errores, los tres en frontend/comun.js: el parámetro de tres bloques catch que no se usaba. Los corregimos.

Build. JavaScript no se compila y el proyecto no usa un empaquetador, así que el lenguaje no exige un build. Igual definimos uno, porque el plan necesita un paquete identificable para desplegar y para revertir. npm run build junta lo que corre en el servidor (server, src y

frontend) en un archivo .tgz dentro de dist, y falla si al paquete le falta un archivo de la aplicación o si incluye tests o documentación. Esta es su salida:

```
> agendaya-tp7-grupo4@1.1.1 build
> node scripts/build.js

Build de agendaya-tp7-grupo4 1.1.1
  Paquete:  dist/agendaya-tp7-grupo4-1.1.1.tgz
  Archivos: 17 (18.0 kB comprimido)
  SHA-1:    a8ad9b6b9052157799e25843e219cf8d0ecd7d6e
```

Tests que reproducen el defecto

El enunciado pide al menos un test nuevo que falle antes del fix y pase después. Agregamos dos archivos, uno por nivel.

El problema para escribirlos. El defecto depende de la zona horaria del dispositivo, y en una computadora configurada con la zona de la agenda no aparece. Lo comprobamos: con el código de la versión 1.1.0 y la computadora en hora de Argentina, las tres verificaciones del test unitario pasan. Un test así no sirve, porque da distinto según dónde se ejecute. Por eso los tests fijan ellos mismos la zona horaria.

Test unitario. El archivo tests/zona-horaria.js hace que los tests de Jest corran siempre con el reloj en La Paz (UTC-4), una hora atrás de la agenda. El proceso de test hace de dispositivo del invitado. El primer test verifica esa condición, para que el escenario no se pierda si alguien cambia la configuración.

Archivo tests/zona-horaria.js:

```
/**
 * Zona horaria de los tests unitarios (globalSetup de Jest).
 *
 * Los tests corren siempre como si el dispositivo estuviera en La Paz (UTC-4),
 * una hora atras de la agenda del administrador (Mendoza, UTC-3).
 *
 * Con el reloj de la maquina en la misma zona que la agenda, un calculo que usa
 * sin querer la zona horaria del dispositivo da bien y el error no se ve. Asi
 * llego INC-0452 a produccion. Fijar aca una zona distinta lo hace visible en
 * cualquier maquina y en el pipeline.
 */
module.exports = async function fijarZonaHoraria() {
  process.env.TZ = "America/La_Paz";
};
```

Archivo tests/unit/m04-confirmacion-zona-horaria.test.js:

```
/**
 * M04 - Texto de la confirmacion de la reserva
 * Responsable: Navarro Matias
 * Funcion: textoConfirmacion
 *
 * Tests de regresion de INC-0452: un invitado con el dispositivo en otra zona
 * horaria veia en la confirmacion el turno corrido una hora.
 *
 * Los tests unitarios corren con el reloj en La Paz (UTC-4), ver
 * tests/zona-horaria.js. La agenda esta en hora de Mendoza (UTC-3), asi que el
 * proceso de test hace de dispositivo del invitado en otra zona horaria.
 */
const { textoConfirmacion } = require("../../src/logica/reservas");
```

```
const reserva = { nombre: "Juan Perez", fecha: "2026-10-13", hora: "10:00" };

describe("textoConfirmacion (INC-0452)", () => {
  it("el dispositivo de prueba esta una hora atras de la agenda (precondicion del
escenario)", () => {
    // Las 10:00 de Mendoza son las 13:00 UTC y las 09:00 de La Paz
    expect(new Date("2026-10-13T13:00:00.000Z").getHours()).toBe(9);
  });

  it("muestra el horario que eligio el invitado aunque su dispositivo este en otra
zona horaria", () => {
    expect(textoConfirmacion(reserva, 60)).toBe("Juan Perez, te esperamos el 2026-10-
13 de 10:00 a 11:00 h.");
  });

  it("calcula el fin del turno con la duracion indicada (caso borde: turno de 45
minutos)", () => {
    expect(textoConfirmacion({ ...reserva, hora: "16:15" }, 45)).toBe(
      "Juan Perez, te esperamos el 2026-10-13 de 16:15 a 17:00 h."
    );
  });

  it("si no se indica la duracion usa la de la agenda, 60 minutos", () => {
    expect(textoConfirmacion(reserva)).toBe("Juan Perez, te esperamos el 2026-10-13 de
10:00 a 11:00 h.");
  });
});
```

Test E2E. Reproduce el incidente como lo reportó Soporte: una reserva desde un teléfono que está en otra zona horaria. Cambia la zona del navegador con el protocolo de depuración de Chrome y usa el tamaño de pantalla de un teléfono. Tiene un caso de control con la zona de la agenda, que pasa con y sin el defecto. En el caso de La Paz verifica primero que la reserva guardada y el correo tengan el horario de la agenda, y después la confirmación en pantalla.

Archivo cypress/e2e/m04-reserva-zona-horaria.cy.js:

```
/**
 * M04 - Booking publico | Test de regresion de INC-0452
 * Reproduce el incidente como lo reporto Soporte: un invitado reserva desde el
 * telefono y su dispositivo esta en otra zona horaria que la agenda del
 * administrador (Mendoza, UTC-3).
 * Responsable: Navarro Matias
 * Trazabilidad: INC-0452 / AYA-M04-RF-03
 */
import { proximoDiaHabil } from "../support/fechas";

/**
 * Cambia la zona horaria del navegador con el protocolo de depuracion de
 * Chrome (sirve en Electron, Chrome y Edge). Con "" vuelve a la del sistema.
 */
function usarZonaHoraria(zona) {
  return cy.wrap(
    Cypress.automation("remote:debugger:protocol", {
      command: "Emulation.setTimezoneOverride",
      params: { timezoneId: zona },
    }),
    { log: false }
  );
}

describe("AgendaYA - M04 Booking: confirmacion y zona horaria del dispositivo (INC-
0452)", () => {
  // Se calcula antes de cambiar la zona del navegador, con el mismo reloj que usa el
```

```
servidor
  const dia = proximoDiaHabil();
  const confirmacionEsperada = "Juan Perez, te esperamos el " + dia + " de 10:00 a
11:00 h.";

  afterEach(() => {
    usarZonaHoraria("");
  });

  function reservarLasDiezDesdeUnTelefonoEn(zona) {
    // Arrange: telefono con esa zona horaria, turno de las 10:00 y datos del invitado
    usarZonaHoraria(zona);
    cy.viewport("iphone-x");
    cy.visit("/reservar.html");
    cy.get('[data-cy="fecha-input"]').type(dia);
    cy.get('[data-cy="slot-1000"]').should("be.enabled").click();
    cy.get('[data-cy="nombre-input"]').type("Juan Perez");
    cy.get('[data-cy="email-input"]').type("juan.perez@ejemplo.com");

    // Act: confirmar la reserva
    cy.get('[data-cy="submit-booking"]').click();
    cy.get('[data-cy="booking-confirmation"]').should("be.visible");
  }

  it("con el telefono en la zona horaria de la agenda confirma el horario elegido", ()
=> {
    reservarLasDiezDesdeUnTelefonoEn("America/Argentina/Mendoza");

    // Assert: caso de control, es el que veiamos nosotros al probar
    cy.get('[data-cy="booking-detalle"]').should("have.text", confirmacionEsperada);
  });

  it("con el telefono en otra zona horaria confirma el mismo horario que eligio el
invitado", () => {
    reservarLasDiezDesdeUnTelefonoEn("America/La_Paz");

    // Assert: la reserva guardada y el correo tienen el horario de la agenda...
    cy.request("/api/reservas").its("body").its(0).its("hora").should("eq", "10:00");
    cy.task("leerCorreos").should("contain", "Turno:   " + dia + " 10:00 h");
    // ...y la confirmacion en pantalla tambien. Aca estaba el defecto: decia "de
09:00 a 10:00 h."
    cy.get('[data-cy="booking-detalle"]').should("have.text", confirmacionEsperada);
  });
});
```

Antes del fix. Sobre el commit que solo tiene los tests, npm run test:unit falla con el síntoma del incidente: esperaba de 10:00 a 11:00 y recibió de 09:00 a 10:00. Los otros 46 tests pasan.

```
FAIL tests/unit/m04-confirmacion-zona-horaria.test.js
  textoConfirmacion (INC-0452)
    ✓ el dispositivo de prueba esta una hora atras de la agenda (precondicion del
escenario) (2 ms)
    × muestra el horario que eligio el invitado aunque su dispositivo este en otra
zona horaria (4 ms)
    × calcula el fin del turno con la duracion indicada (caso borde: turno de 45
minutos)
    × si no se indica la duracion usa la de la agenda, 60 minutos (1 ms)

  ● textoConfirmacion (INC-0452) › muestra el horario que eligio el invitado aunque su
dispositivo este en otra zona horaria

    expect(received).toBe(expected) // Object.is equality

    Expected: "Juan Perez, te esperamos el 2026-10-13 de 10:00 a 11:00 h."
    Received: "Juan Perez, te esperamos el 2026-10-13 de 09:00 a 10:00 h."
```

```
  [...]

Test Suites: 1 failed, 5 passed, 6 total
Tests:       3 failed, 46 passed, 49 total
```

Después del fix. Sobre el commit de la corrección pasan los 49:

```
PASS tests/unit/m04-confirmacion-zona-horaria.test.js
  textoConfirmacion (INC-0452)
    ✓ el dispositivo de prueba esta una hora atras de la agenda (precondicion del
escenario)
    ✓ muestra el horario que eligio el invitado aunque su dispositivo este en otra
zona horaria
    ✓ calcula el fin del turno con la duracion indicada (caso borde: turno de 45
minutos)
    ✓ si no se indica la duracion usa la de la agenda, 60 minutos

Test Suites: 6 passed, 6 total
Tests:       49 passed, 49 total
```

Tests	Antes del TP7	Con el hotfix
Unitarios (Jest)	45	49
E2E (Cypress)	10	12

Protección de ramas

main y develop están protegidas con la misma regla. Es la que hace que el pipeline bloquee de verdad: sin los dos jobs en verde y sin una aprobación, el botón de merge no se habilita.

Opción	Valor
Require a pull request before merging	Sí
Required approvals	1
Dismiss stale pull request approvals when new commits are pushed	Sí
Require status checks to pass before merging	Sí
Status checks requeridos	Formato, linter, tests unitarios y build Tests E2E (Cypress)
Require branches to be up to date before merging	Sí
Do not allow bypassing the above settings	Sí. Vale también para los administradores.

No se puede hacer push directo a ninguna de las dos ramas, y GitHub no permite que el autor de un Pull Request lo apruebe.

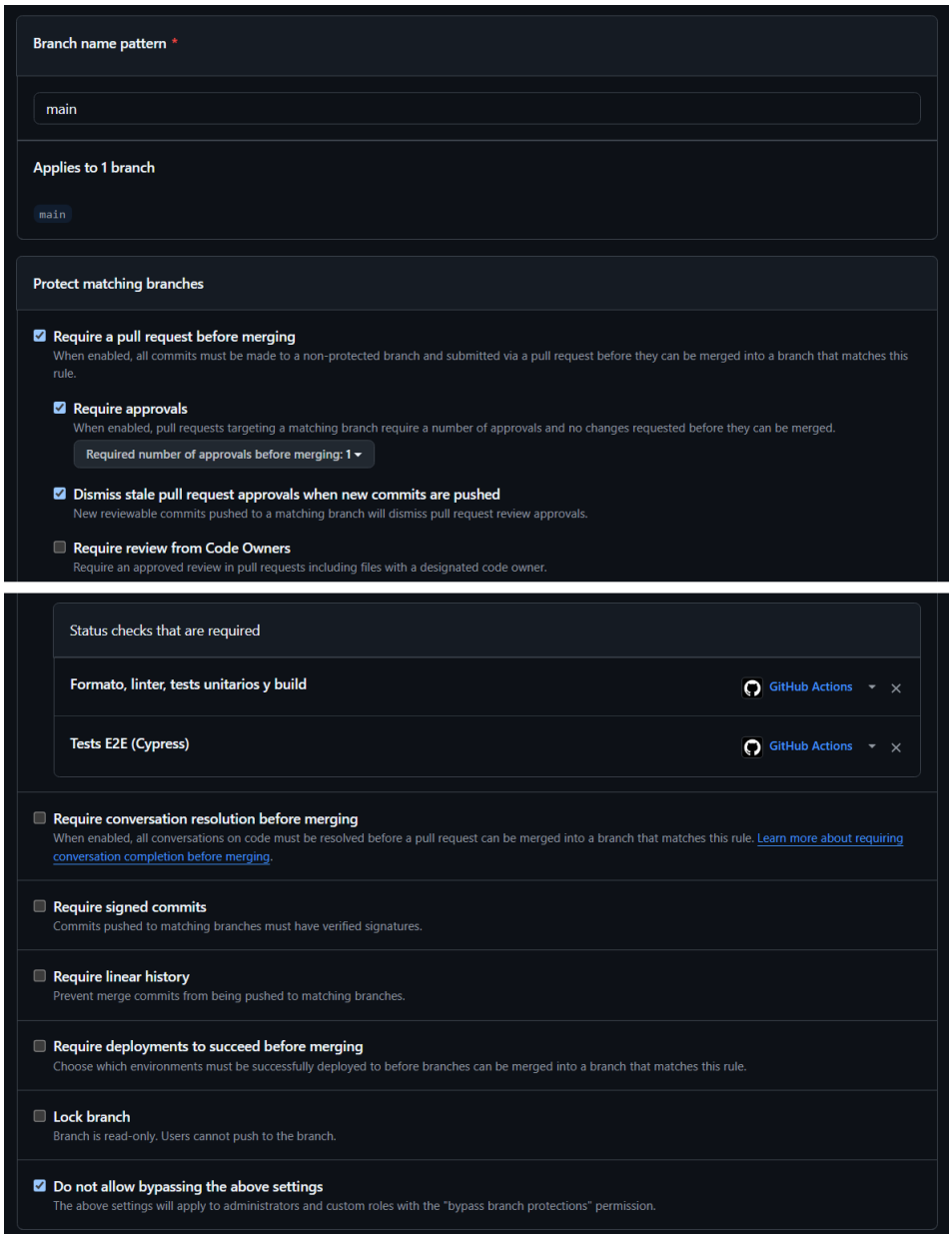


Figura 4 – Regla de protección de la rama main: Pull Request obligatorio con una aprobación, los dos checks requeridos y sin excepciones para administradores. La de develop es igual.

Ejecuciones del pipeline

Estas son las ejecuciones que generó el recorrido del hotfix:

N.º	Disparador	Commit	Resultado	Evidencia
1	Push a main	Versión 1.1.0, con el defecto	Verde (1 min 22 s). Los tests que existían no lo detectan.	Figura 3

N.º	Disparador	Commit	Resultado	Evidencia
2	Pull Request del hotfix hacia main	Tests que reproducen INC-0452	Rojo. Falla el paso de tests unitarios (3 de 49) y el job de tests E2E (1 de 12). Formato, linter y build quedan en verde.	Figuras 5, 6 y 7
3	Pull Request del hotfix hacia main	Corrección (v1.1.1)	Verde (56 s). Pasan los 49 tests unitarios y los 12 E2E.	Figuras 8 y 9
4	Push a main	Commit de merge del hotfix (494d376), etiquetado v1.1.1	Verde (1 min 21 s)	—
5	Pull Request del hotfix hacia develop	Corrección (v1.1.1)	Verde (1 min 16 s)	Figura 10
6	Push a develop	Commit de merge del hotfix (17d7041)	Verde (1 min 3 s)	—

En la ejecución 2 se ve que cada verificación es un paso separado: fallan los tests unitarios, mientras que el formato, el linter y el build quedan en verde. El job de tests E2E también falla, por el test nuevo del escenario del incidente. Con los checks en rojo GitHub bloquea el merge (Figura 5).

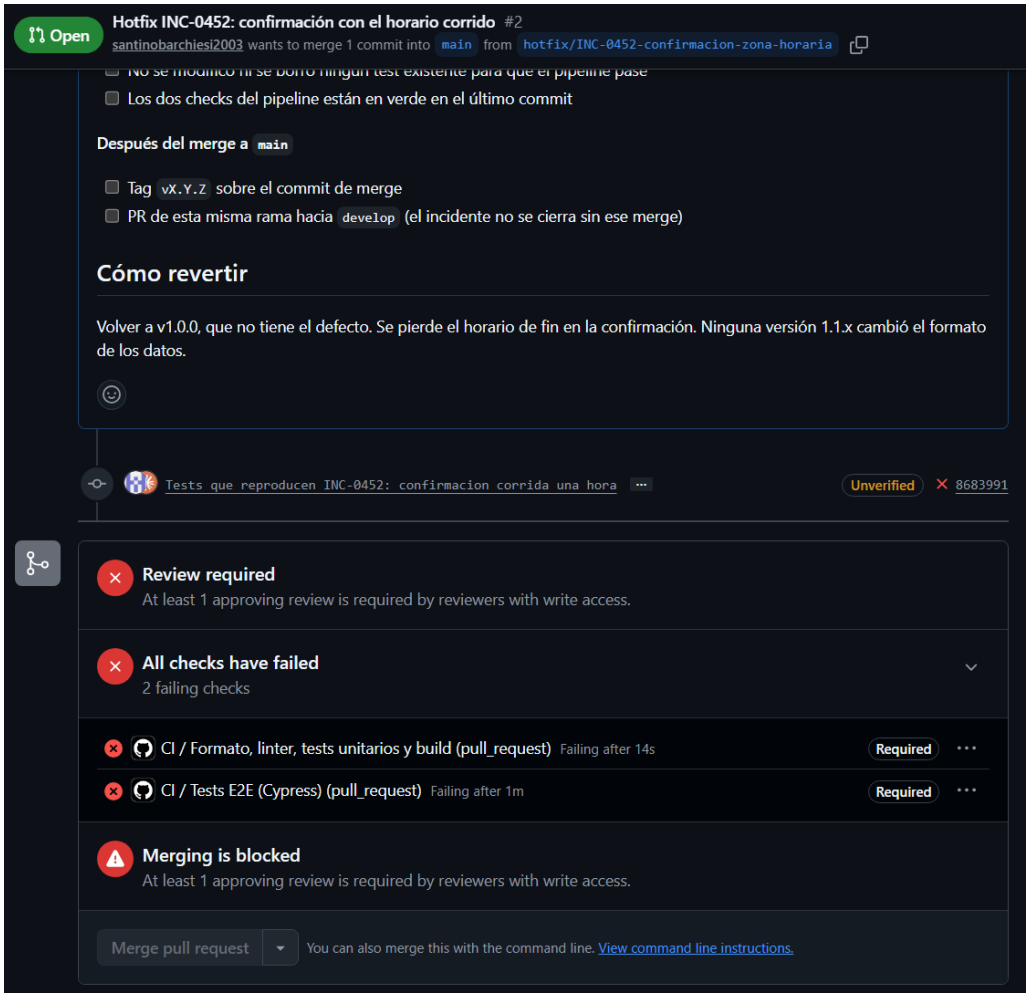


Figura 5 – Pull Request del hotfix con el commit de los tests: fallan los dos checks obligatorios, falta la revisión y el merge queda bloqueado.

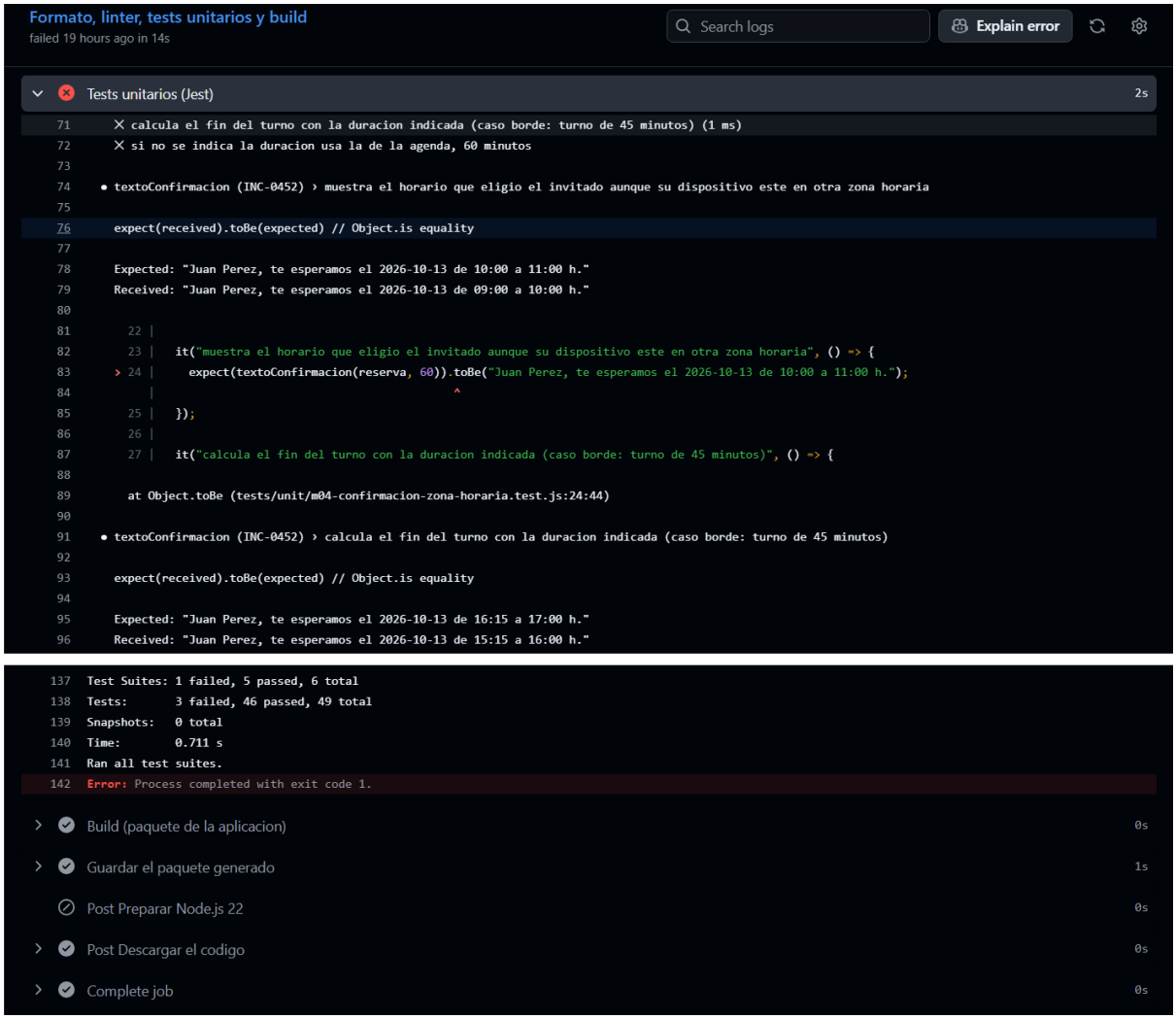


Figura 6 – Detalle de la ejecución fallida: el paso Tests unitarios (Jest) con el horario esperado y el recibido. Abajo, el resumen (3 tests fallan y 46 pasan) y el paso de build en verde.

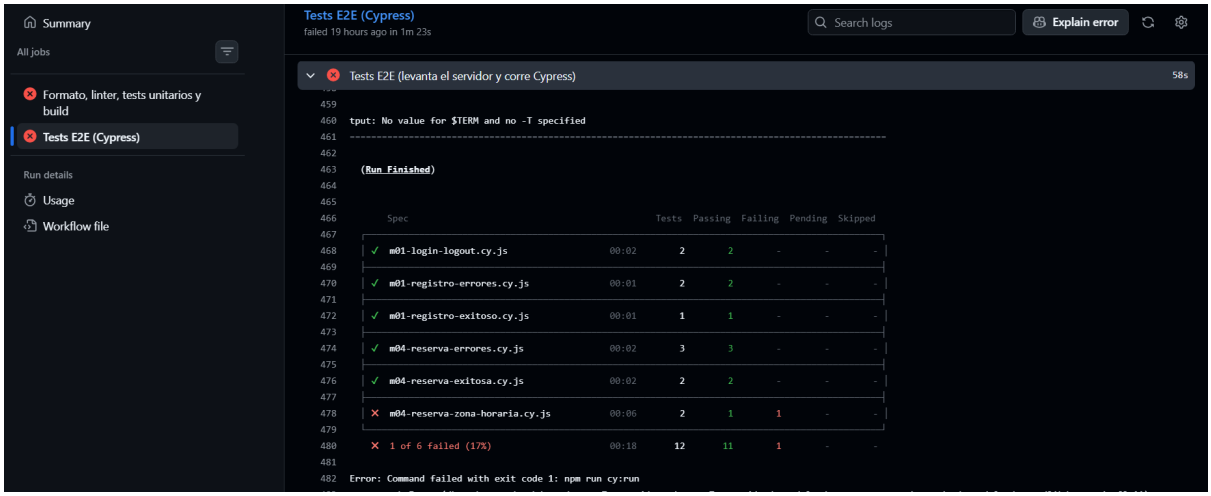


Figura 7 – Job Tests E2E (Cypress) de la misma ejecución: de 12 tests falla uno, el del teléfono en otra zona horaria.

Con el commit de la corrección los dos jobs pasan, el revisor aprueba y el Pull Request se puede mergear:

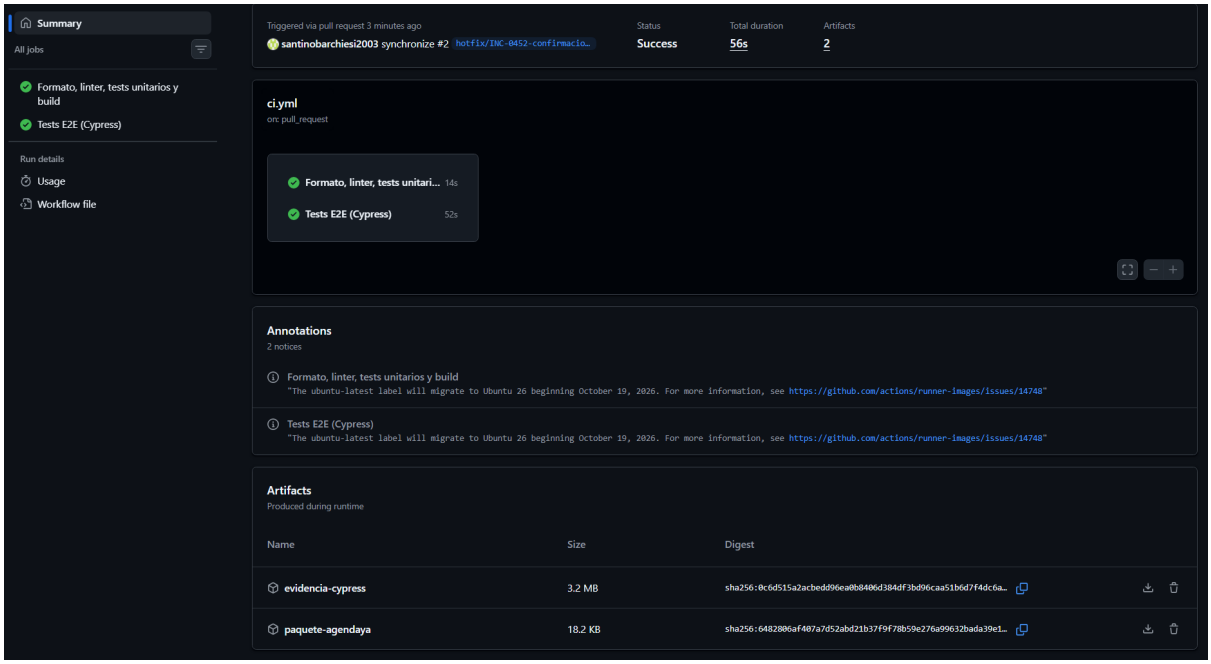


Figura 8 – Ejecución del Pull Request sobre el commit de la corrección: los dos jobs en verde en 56 segundos, con el paquete y la evidencia de Cypress como artefactos.

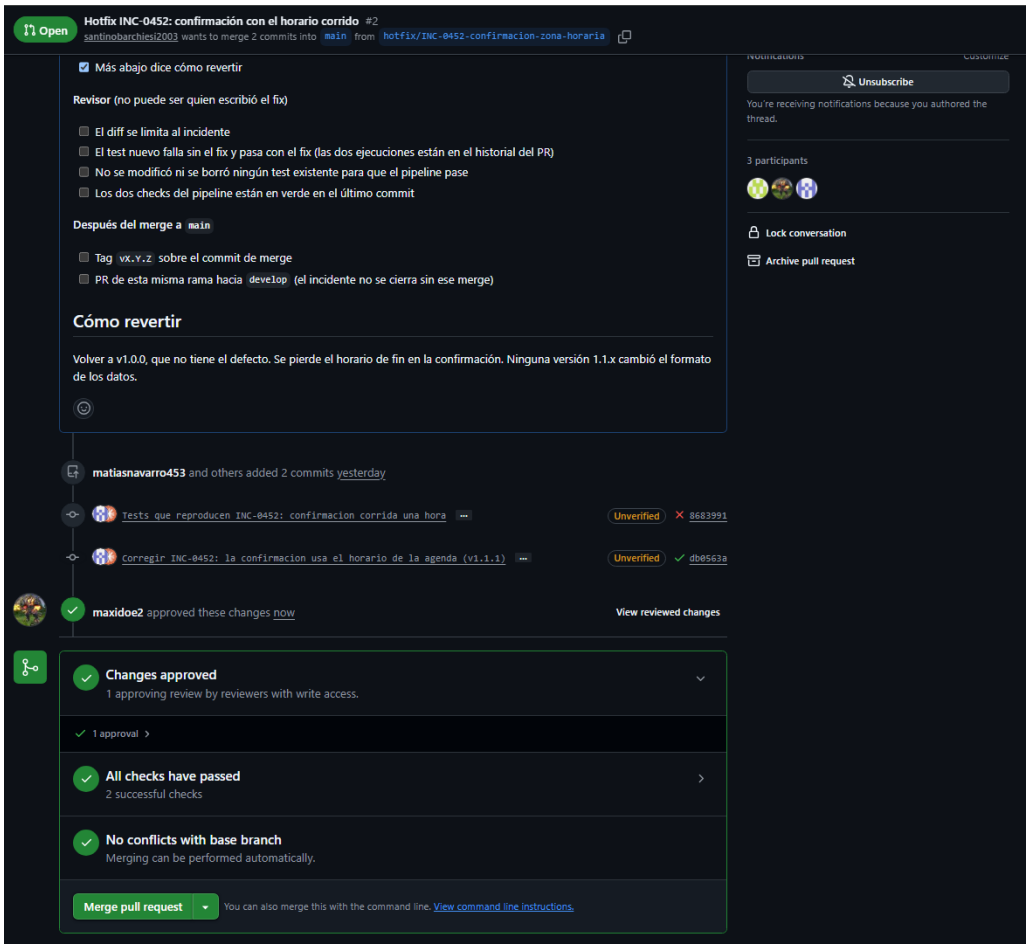


Figura 9 – Pull Request del hotfix antes del merge: el commit de los tests en rojo, el de la corrección en verde, la aprobación del revisor y el merge habilitado.

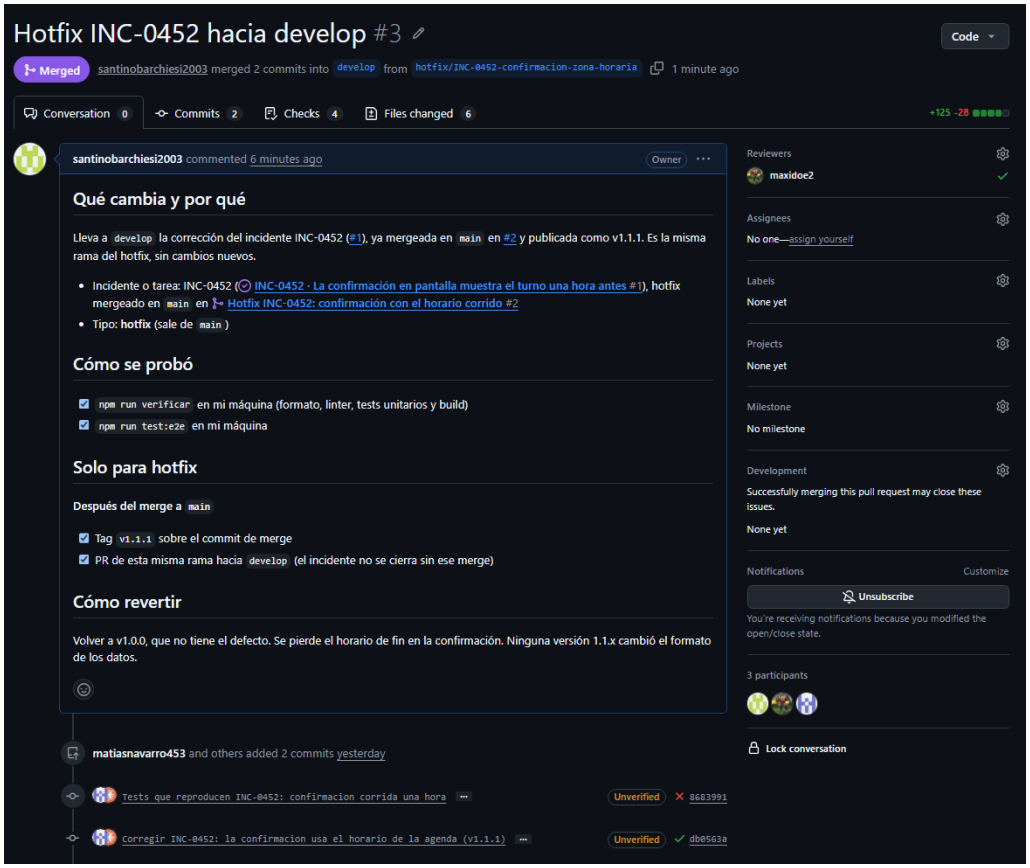


Figura 10 – Pull Request de la misma rama hacia develop, aprobado por el revisor y mergeado.

Consistencia entre el plan y el pipeline

El enunciado pide que toda rama por la que pasa el hotfix esté cubierta por el pipeline y que las aprobaciones del plan se reflejen en el repositorio.

Lo que dice el plan	Dónde se cumple en el repositorio
El hotfix entra a main por Pull Request.	El workflow corre ante pull_request hacia main, y main no acepta push directo.
El hotfix vuelve a develop por Pull Request.	El workflow corre ante pull_request hacia develop, que tiene la misma protección.
No hay otra rama de destino para un hotfix.	No hay más ramas que cubrir. La rama hotfix/* es el origen de los dos Pull Request.
Una aprobación de alguien que no escribió la corrección.	Protección de rama: una aprobación requerida. GitHub no deja que el autor apruebe su Pull Request.
La aprobación vale para lo que se revisó.	Protección de rama: la aprobación se descarta si entra un commit nuevo.
Los dos jobs en verde para poder mergear.	Protección de rama: los dos checks son requeridos, también para los administradores.

Lo que dice el plan	Dónde se cumple en el repositorio
Lo que se despliega es el paquete del pipeline.	El primer job guarda el paquete del build como artefacto de la ejecución sobre main.
El Tech Lead aprueba el despliegue a producción.	Queda como comentario en el issue del incidente. No está automatizado en el repositorio: ver la justificación del nivel avanzado.

Justificación de los niveles del pipeline

N.º	Requisito	Estado	Cómo, o por qué no
1	Ejecutar los tests unitarios	Abordado	Paso Tests unitarios (Jest): 49 tests.
2	Linter	Abordado	Paso Linter (ESLint), también sobre los scripts de las pantallas.
3	Formato	Abordado	Paso Formato (Prettier).
4	Build	Abordado	JavaScript no lo requiere. Igual hay un paso Build, que arma y verifica el paquete que se despliega.
5	Test nuevo que reproduce el defecto	Abordado	Un archivo de tests unitarios y uno de tests E2E. Fallan sobre el commit anterior al fix y pasan con el fix.
6	Ejecución automática ante Pull Requests hacia develop y main	Abordado	Disparador pull_request para las dos ramas. El plan no define otra rama de destino para el hotfix.
7	Protección de develop y main	Abordado	Regla de protección con los dos jobs como checks requeridos y una aprobación.
8	Tests E2E de Cypress	Abordado	Job Tests E2E (Cypress): los 10 tests del TP6 y los 2 nuevos.
9	Despliegue automático a desarrollo en Firebase Hosting	No abordado	Ver abajo.
10	Despliegue a producción con aprobación manual y Environments	No abordado	Ver abajo.

Por qué no abordamos el nivel avanzado

Firebase Hosting publica archivos estáticos. Nuestra aplicación no lo es: las pantallas dependen del servidor Express para iniciar sesión, consultar la agenda, validar y guardar la reserva y registrar los correos. Publicar solo la carpeta frontend dejaría pantallas que no pueden iniciar sesión ni reservar.

Para desplegarla en Firebase habría que mover el servidor a Cloud Functions o a Cloud Run, que necesitan el plan pago de Firebase, y reemplazar la persistencia en un archivo JSON por una base de datos, porque esos servicios no conservan el disco entre

ejecuciones. Eso es rediseñar la aplicación, y queda fuera de un trabajo sobre mantenimiento.

Lo que dejamos en su lugar:

- El plan define el despliegue a producción como un paso manual con aprobación del Tech Lead (sección 6) y dice cómo queda registrado.
- El pipeline genera y guarda el paquete que se desplegaría, así que el paso manual no arma nada a mano.
- Lo anotamos como riesgo en las lecciones aprendidas: el paso que llega a producción es el único que no está automatizado.

Si lo abordáramos, el job de despliegue a producción usaría un environment de GitHub llamado production, con el Tech Lead como revisor requerido. Es la misma persona que aprueba en la sección 6 del plan, así que la configuración quedaría consistente con el plan.

Lecciones aprendidas

Seguimos las categorías del anexo de lecciones aprendidas del TP N.º 1.

Qué funcionó

1. **Escribir primero el test que falla.** El primer commit del hotfix tiene solo los tests, y el pipeline quedó en rojo con el mismo síntoma que reportó Soporte: esperaba de 10:00 a 11:00 y recibió de 09:00 a 10:00. Recién después escribimos la corrección. Así sabemos que el test detecta ese defecto y no otra cosa.
2. **Que la protección de ramas no tenga excepciones.** Con el SLA corriendo, la tentación es mergear directo. Como la regla vale también para los administradores, el camino corto no existe y la revisión se hace igual.
3. **Separar el formateo en un commit propio.** Formatear todo el código tocó 13 archivos. Al ir solo, los commits siguientes muestran únicamente cambios reales y la revisión del hotfix es corta.
4. **La lógica separada de las pantallas, que venía del TP6.** Como el texto de la confirmación lo arma una función de src/logica, pudimos reproducir el defecto con un test unitario que corre en milisegundos, sin abrir un navegador.

Qué falló

1. **El pipeline pasó en verde con el defecto adentro.** Los 45 tests unitarios y los 10 E2E pasaban con la versión 1.1.0. La función nueva no tenía tests y el E2E de la reserva verificaba el día de la confirmación pero no la hora. Un pipeline en verde dice que pasaron los tests que existen, no que el cambio esté probado.
2. **Todo se probaba desde la misma zona horaria.** Con la computadora en la zona de la agenda el defecto no aparece. Hizo falta que los tests fijen una zona distinta para verlo.
3. **El síntoma no era la causa.** El reporte decía "con el celular" y el celular no tenía nada que ver: lo que cambiaba era la zona horaria del dispositivo. Empezar por el diseño para pantallas chicas habría gastado tiempo del SLA.

Herramientas

1. **GitHub Actions.** Con dos jobs en paralelo y pasos con nombre propio, la ejecución se lee sola: se ve qué verificación falló sin abrir el registro. La condición para que un paso corra aunque falle el anterior hay que escribirla, porque por defecto el job se corta en el primer error.
2. **ESLint.** Encontró 3 errores en código que ya funcionaba y, al corregir el incidente, marcó las constantes y la función que habían quedado sin uso. Hubo que sumarle un complemento para que analice los scripts que están dentro del HTML.
3. **Prettier y los finales de línea.** Prettier exige finales de línea LF, y Git para Windows convierte los archivos a CRLF al clonar. Sin .gitattributes el control de formato falla en la computadora y pasa en el pipeline.

4. **Emular la zona horaria.** Jest permite fijar la zona del proceso antes de correr los tests, y Cypress puede cambiar la del navegador con el protocolo de depuración de Chrome. Sin eso el escenario del incidente no se podía automatizar.

Riesgos

1. **Un hotfix saltea parte del camino normal.** No pasa por develop antes de llegar a producción. Lo compensamos exigiéndole el mismo pipeline, la misma revisión y la validación en staging.
2. **Que el fix se pierda en la próxima versión.** Si el cambio queda solo en main, la próxima versión que salga de develop vuelve a traer el defecto. Por eso el incidente no se cierra hasta mergear el segundo Pull Request, y por eso el test de regresión viaja con el fix.
3. **Un equipo de tres depende de personas puntuales.** Si el revisor no está, la revisión no la puede hacer el autor. Definimos suplentes y una salida que no necesita una revisión nueva: volver a la última versión sana.
4. **El despliegue a producción sigue siendo manual.** Es el paso más delicado y el único que no está automatizado.
5. **Tests que dependen del entorno.** Un test que da distinto según la zona horaria de la máquina puede pasar en la computadora del desarrollador y fallar en el pipeline, o al revés.

Defectos técnicos y cómo se solucionaron

Defecto	Causa	Solución
INC-0452: confirmación con el horario corrido.	textoConfirmacion mostraba el horario con getHours() y getMinutes(), que dependen de la zona horaria del dispositivo.	Usar la hora de la agenda tal como está guardada y calcular el fin sumando minutos (v1.1.1).
3 errores de ESLint en frontend/comun.js.	El parámetro de tres bloques catch estaba declarado y no se usaba.	Se quitó el parámetro.
Código sin uso después de la corrección.	Quedaron las constantes DESFASE_AGENDA y MS_POR_MINUTO y la función horaDe.	Los marcó el linter y se borraron en el mismo commit.

Defecto	Causa	Solución
El pipeline no arrancaba solo en el primer repositorio que creamos.	No la encontramos. El workflow era válido y corría al lanzarlo a mano, pero GitHub no generaba ejecuciones ante el push ni ante el Pull Request. En un repositorio de prueba de la misma cuenta sí arrancaba.	Creamos el repositorio de nuevo y subimos primero solo main, para comprobar que el push disparaba el pipeline antes de seguir. Las capturas son de ese repositorio.
Cypress no arrancaba desde la terminal de VS Code.	VS Code define la variable ELECTRON_RUN_AS_NODE en sus terminales y Cypress no inicia con esa variable.	Quitar la variable para ese comando. En GitHub Actions no pasa.

Uso de IA

Usamos Claude (Anthropic), un asistente de IA que puede leer repositorios, escribir código y ejecutar comandos. Le pasamos el enunciado del TP7, los informes del TP5 y del TP6 y los dos repositorios, con esta consigna:

> "Realiza el tp7, te dejo el 5 y 6 resueltos y sus repositorios, si necesitas algun dato me avisas"

En esa primera versión la herramienta usó el incidente de ejemplo del enunciado. Cuando supimos que el incidente lo tenía que definir cada equipo, se lo indicamos con una segunda consigna:

> "ya hable con un compañero y me dijo que el incidente hay que inventarlo, que mas hay que hacer?"

Con eso reemplazó el incidente por uno propio, INC-0452, sobre el mismo defecto: reescribió la ficha del incidente y el análisis del síntoma, y cambió el nombre de la rama, los commits y los tests.

Qué generó. La configuración de Prettier y de ESLint, el script de build, el workflow del pipeline, las plantillas, la ficha del incidente, el cambio que introduce el defecto, los tests que lo reproducen, la corrección y la redacción de este informe.

Qué decidió.

Decisión	Qué eligió
Qué incidente plantear	Uno de M04 sobre la zona horaria del dispositivo, con el ejemplo del enunciado como modelo.
Cómo inyectar el defecto	Un cambio chico en la confirmación (mostrar el horario de fin) que usa el reloj del dispositivo. Produce el síntoma del incidente y no rompe ningún test que ya existiera.
Cómo reproducirlo en un test	Fijar la zona horaria en los tests, porque con la computadora en la zona de la agenda el defecto no aparece.
Qué niveles del pipeline abordar	El obligatorio y el deseable. Explicó por qué Firebase Hosting no sirve para una aplicación con servidor.
Roles del equipo	Los propuso según lo que hizo cada uno en el TP5 y el TP6.

Qué no pudo hacer en esa primera etapa. Trabajaba en un entorno propio, sin acceso a nuestra cuenta de GitHub. No pudo ejecutar Cypress, porque la descarga estaba bloqueada, y verificó las pantallas repitiendo los pasos de los tests con otro navegador automatizado. Tampoco pudo crear el repositorio ni generar las ejecuciones del pipeline, así que entregó el repositorio armado y una guía con los pasos que faltaban.

Cómo se hicieron los pasos en GitHub. Instalamos Claude Code en Visual Studio Code, en la computadora de Santino, y le indicamos que siguiera esa guía sin modificar archivos y

que se detuviera ante cualquier resultado distinto al esperado. La herramienta instaló Node.js y GitHub CLI, corrió los tests E2E con Cypress (12 en verde con la corrección, y 11 en verde y 1 en rojo sin ella), creó el repositorio, configuró la protección de ramas, registró el incidente, subió los commits, abrió los dos Pull Request, hizo los merges y creó el tag v1.1.1. También detectó los dos problemas de la tabla de defectos técnicos y se detuvo en cada uno. Cada vez que se detuvo, le pasamos el resultado a la primera sesión de Claude, que indicó cómo seguir.

Qué hicimos nosotros. Iniciamos sesión en GitHub, pasamos los resultados de una sesión a la otra, aprobamos los dos Pull Request desde la cuenta de Máximo y tomamos las capturas de este informe.